

AgentT1000: agente híbrido para Connect-4

Fundamentos de Inteligencia Artificial - Reto Connect-4 | Agente final en `groups/Group C/policy.py` | Branch: <https://github.com/Leo07102004/Fundamentos-IA-Final-Project-CruzMontoyaPortocarrero/tree/Cruz>

1. Idea principal y arquitectura del agente

AgentT1000 se diseñó como un agente híbrido: primero toma decisiones tácticas deterministas para evitar errores inmediatos; después consulta conocimiento aprendido offline mediante una tabla de Q-values; y, cuando no existe evidencia suficiente para un estado-accion, usa Monte Carlo Tree Search (MCTS) como respaldo. Esta combinación permite que el agente sea explicable, rápido en posiciones conocidas y robusto en posiciones nuevas.

1. Victoria inmediata Si existe una columna que conecta 4 fichas, se juega de una vez.	2. Bloqueo rival Si el oponente puede ganar en el siguiente turno, se bloquea esa amenaza.	3. Filtro de seguridad Descarta jugadas que dejan una victoria inmediata al rival.	4. Q-values o MCTS Usa la tabla aprendida si la accion fue visitada al menos 3 veces; si no, usa MCTS.
--	--	--	--

Componente	Uso en AgentT1000	Configuracion/criterio
Reglas tacticas	Ganar, bloquear y evitar jugadas suicidas.	Se evalúan columnas legales antes de buscar.
Q-values offline	Prioriza acciones aprendidas por simulacion.	Usa $Q(s,a)$ si <code>returns_count >= 3</code> .
MCTS fallback	Explora estados no cubiertos por la tabla.	350 iteraciones o 0.20 s por jugada.
Preferencia central	Desempata hacia columnas centrales.	Bonus proporcional a cercanía a columna 3.

2. Entrenamiento offline usado para generar q_values.pkl

El entrenamiento offline se hizo con **First-Visit Monte Carlo (FVMC)** sobre partidas simuladas. Se generaron 20,000 episodios con una política de comportamiento epsilon-greedy ($\epsilon = 0.20$). En cada turno, el simulador primero busca una victoria inmediata, luego un bloqueo al rival y, si no existe ninguna de esas acciones, explora o explota usando Q-values con un pequeño sesgo hacia el centro.

Paso	Descripcion técnica
Generar episodio	La partida empieza con tablero vacío; alternan jugadores -1 y 1 hasta victoria o empate.
Registrar estado-accion	Cada jugada guarda (board_key, accion, jugador). board_key incluye el tablero plano y el jugador actual.
Calcular retorno	Al final: $G = +1$ si ganó el jugador asociado a esa accion, $G = -1$ si perdió y $G = 0$ si fue empate.
Actualizar $Q(s,a)$	Solo primera visita del par estado-accion en el episodio: $Q(s,a) = \text{promedio de retornos observados}$.
Guardar modelo	Se exporta q_values.pkl con q_values, returns_count y numero de episodios.

En ejecución, la tabla no reemplaza completamente al razonamiento: el agente usa los Q-values solo cuando tienen un mínimo de evidencia (al menos tres visitas). Si el estado-accion no aparece o tiene baja cobertura, el agente recurre a MCTS. Por eso el enfoque offline se integra como una memoria de experiencia, no como una política rígida.

Validación experimental y conclusiones

Todos los experimentos se ejecutaron con $n = 100$ partidas por escenario. Se midieron ganador, porcentaje de victorias, duración promedio, tiempo por jugada y movimientos ilegales.

Contra Random

100% / 100%

Victorias como rojo y amarillo. Sin derrotas.

Contra T600

85% / 88%

Victorias como rojo y amarillo frente a un rival con MCTS.

Autojuego

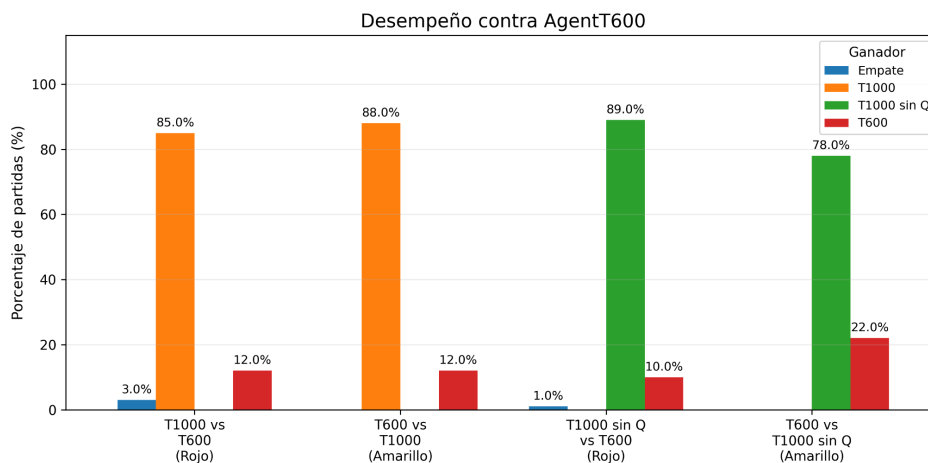
71% amarillo

Amarillo gana 71%, rojo 24%, empate 5%.

3. Resultados empíricos principales

Escenario	T1000 completo	Version sin Q-values	Insight
vs Random - rojo	100% victorias	100% victorias	Ambas versiones cumplen el requisito mínimo.
vs Random - amarillo	100% victorias	100% victorias	El agente no pierde contra el aleatorio en ningún color.
vs T600 - rojo	85% victorias, 12% derrotas, 3% empates	89% victorias, 10% derrotas, 1% empates	Los Q-values no mejoran todo de forma uniforme.
vs T600 - amarillo	88% victorias, 12% derrotas	78% victorias, 22% derrotas	Los Q-values aportan +10 pp cuando juega segundo.
Autojuego	71% amarillo, 24% rojo, 5% empate	-	El desempeño depende del color y de la secuencia de estados.

Figura 1. Desempeño contra AgentT600



La comparación contra T600 permite evaluar el agente frente a un rival más fuerte que Random. La versión completa mantiene alto desempeño: 85% como rojo y 88% como amarillo. La diferencia con la versión sin Q-values evidencia que el aprendizaje offline ayuda especialmente cuando el agente juega como amarillo.

4. Conclusiones del análisis

- AgentT1000 cumple y supera el requisito mínimo: nunca pierde contra Random y gana 100% de las partidas en ambos colores.
- Contra T600 conserva desempeño alto: 85% como rojo y 88% como amarillo.
- No se observaron movimientos ilegales en los experimentos.
- Los Q-values reducen dependencia de MCTS en estados conocidos y mejoran el resultado como amarillo contra T600; sin embargo, su efecto no es uniforme, lo que revela un área de mejora.

5. Propuesta de mejora

- Aumentar el número y diversidad de episodios offline para cubrir más estados intermedios y finales.
- Entrenar con oponentes mixtos, no solo comportamiento propio, para reducir sesgos por color.
- Hacer tuning sistemático de recursos MCTS (iteraciones/tiempo) y del umbral `returns_count >= 3`.
- Agregar evaluación de sensibilidad para seleccionar la mejor combinación entre cobertura de Q-values y costo computacional.